

Capítulo 4

Segurança no .NET Core 8

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/IPC - JB



4.1. Segurança em Aplicações Web

Programação Web – 2024/2025 – LEI DAPOO-PL – ISEC/IPC - JB

CONCEITOS GERAIS

- **Autenticação**



  - Processo de verificação de identidade de um utilizador ou computador
    - Quem é? Como o provar?
    - Verifica se utilizador é realmente quem diz ser
    - Credenciais podem ser especificadas através de um código de acesso (*password*), um *token* externo, um smart card, …

Programação Web – 2024/2025 – LEI+CE+PL – ISEC/IPC - JB

CONCEITOS GERAIS

- **Autorização**
  - Processo para determinar o que é que o utilizador pode aceder num determinado computador, rede, …
    - Quais as permissões?
    - Tem acesso a este conteúdo?
    - Tem permissões para ver o conteúdo desta página?



Programação Web – 2024/2025 – LEI D+GE+PL – ISEC/IPC - JB

EVOLUÇÃO: IDENTIDADE EM ASP.NET



- 2005: *ASP.NET Membership*
  - Armazenamento e gestão de dados da identidade
  - *SQL Server*
- 2011: *Simple Membership*
  - Sistema de *Membership* para modelo *ASP.NET Web Pages*
  - Não é possível usar com o *OWIN* (*Open Web Interface for .NET*)

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/DEC - JB

EVOLUÇÃO: IDENTIDADE EM ASP.NET



- 2012: *Universal Providers Membership*
  - Passou a permitir armazenamento de dados adicionais
  - Suporte à associação de fornecedores externos
  - Permitiu armazenamento da informação *membership* no *Windows Azure SQL Database*
  - Permitiu o uso do *Membership* com a *EF*
- Após 2013: ***ASP.NET Identity***
- **Actualmente: *ASP.Net Core Identity Framework***

Programação Web – 2024/2025 – LEI+LD+CE+... – ISEC/DEC - JB

ASP.NET CORE IDENTITY: CARACTERÍSTICAS



- **Autenticação** e **Autorização** para aplicações *ASP.NET* Core
  - Suporta *ASP.NET Core, MVC, Web API, Web Forms, SignalR, Web Pages, Componentes .Net Core 8*
  - *No .Net Core 8 foi estendido à utilização em API*
- Permite efetuar a manipulação de utilizadores, *profiles*, processo de login/logout, criação de perfis, *claims*, etc…
  - Permite manter as contas do utilizador numa base de dados local ou num armazenamento de dados externo



Programação Web – 2024/2025 – LEI D+CE+EI – ISEC/IPC - JB

ASP.NET CORE IDENTITY: CARACTERÍSTICAS

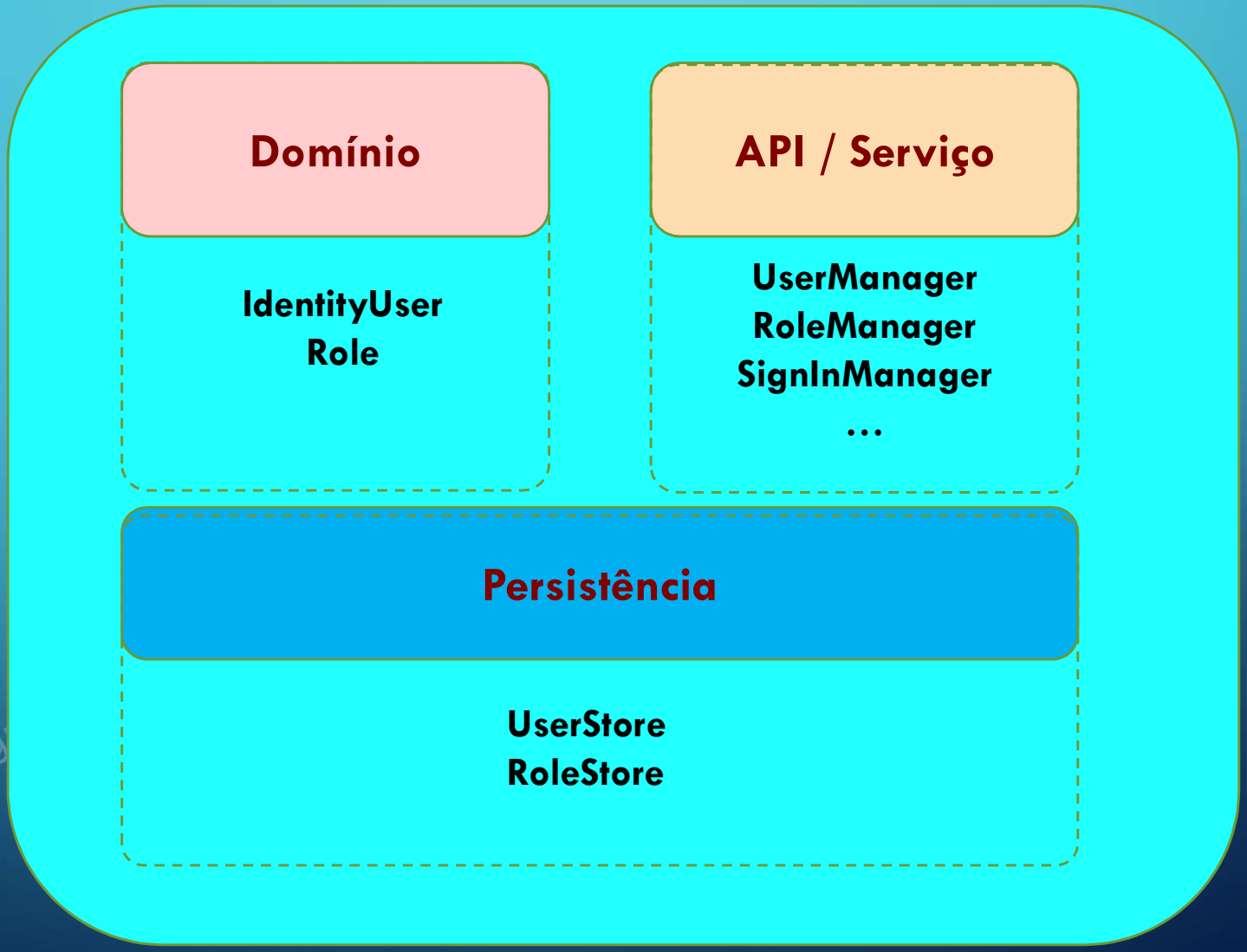


- Facilita escrita de testes unitários
- Permite login externo (através do ***OAuth***)
  - Suporta *Facebook*, *Google*, *Microsoft*, *Twitter*, *accounts*
- Suporta o *middleware* ***OWIN***
  - Pode executar num servidor que não o *IIS* da *Microsoft*
- Disponível através do gestor de bibliotecas ***NuGet***

Programação Web – 2024/2025 – LEI DEI – CPLP – ISEC/IPC - JB

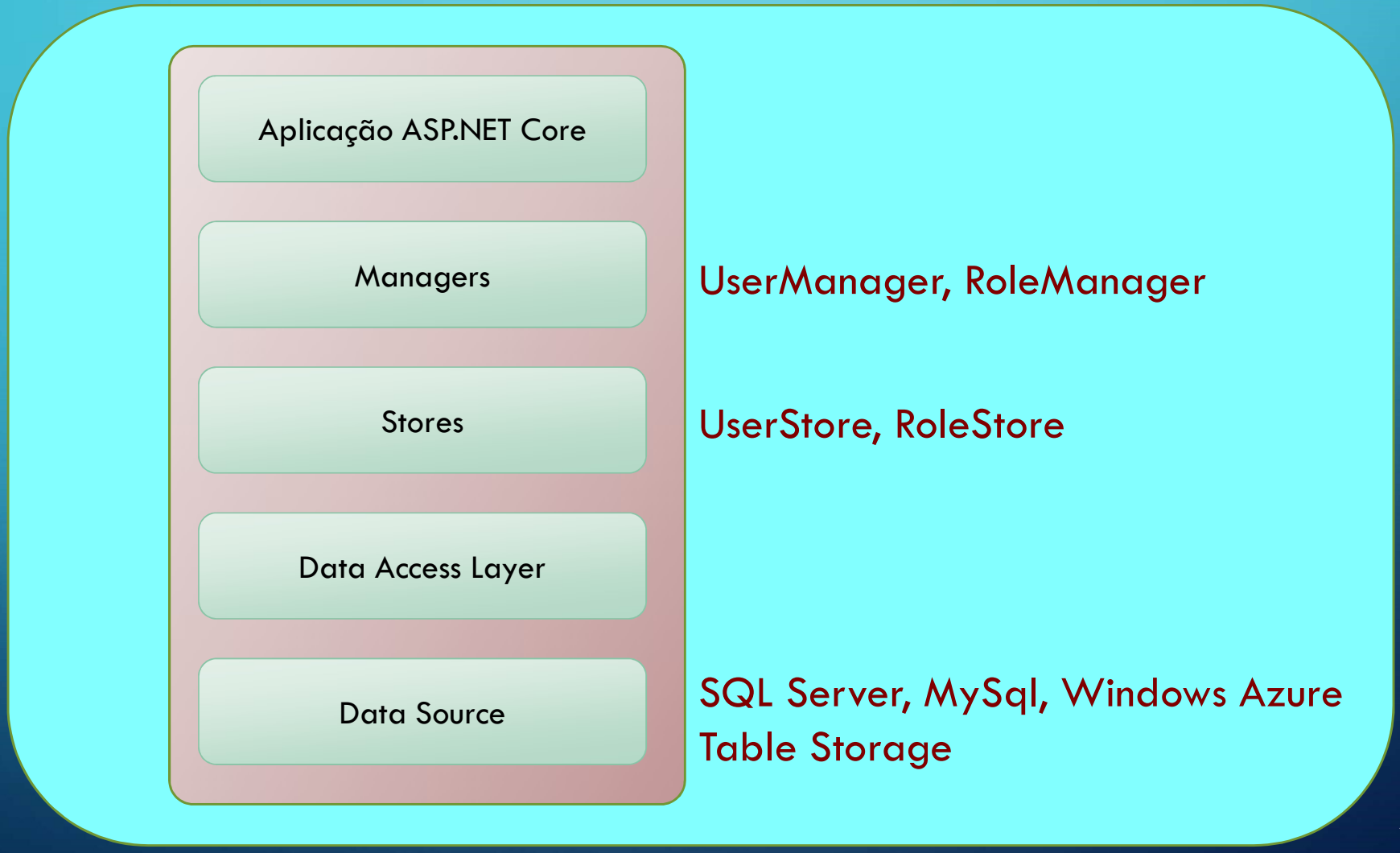
ASP.NET CORE IDENTITY: ARQUITETURA





ASP.NET CORE IDENTITY: ARQUITETURA

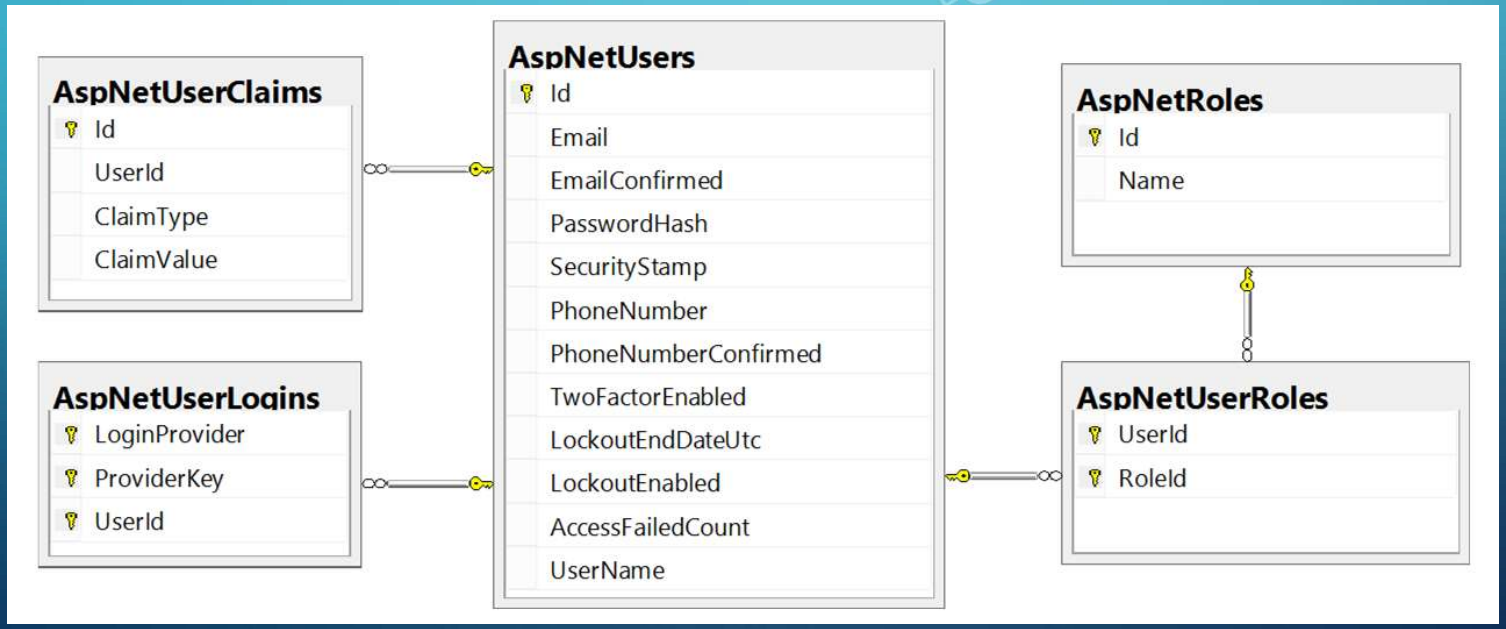


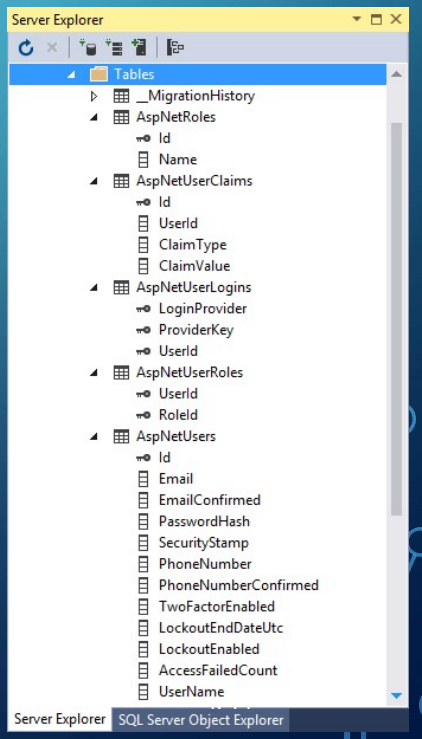


ASP.NET CORE IDENTITY: MODELO DE DADOS

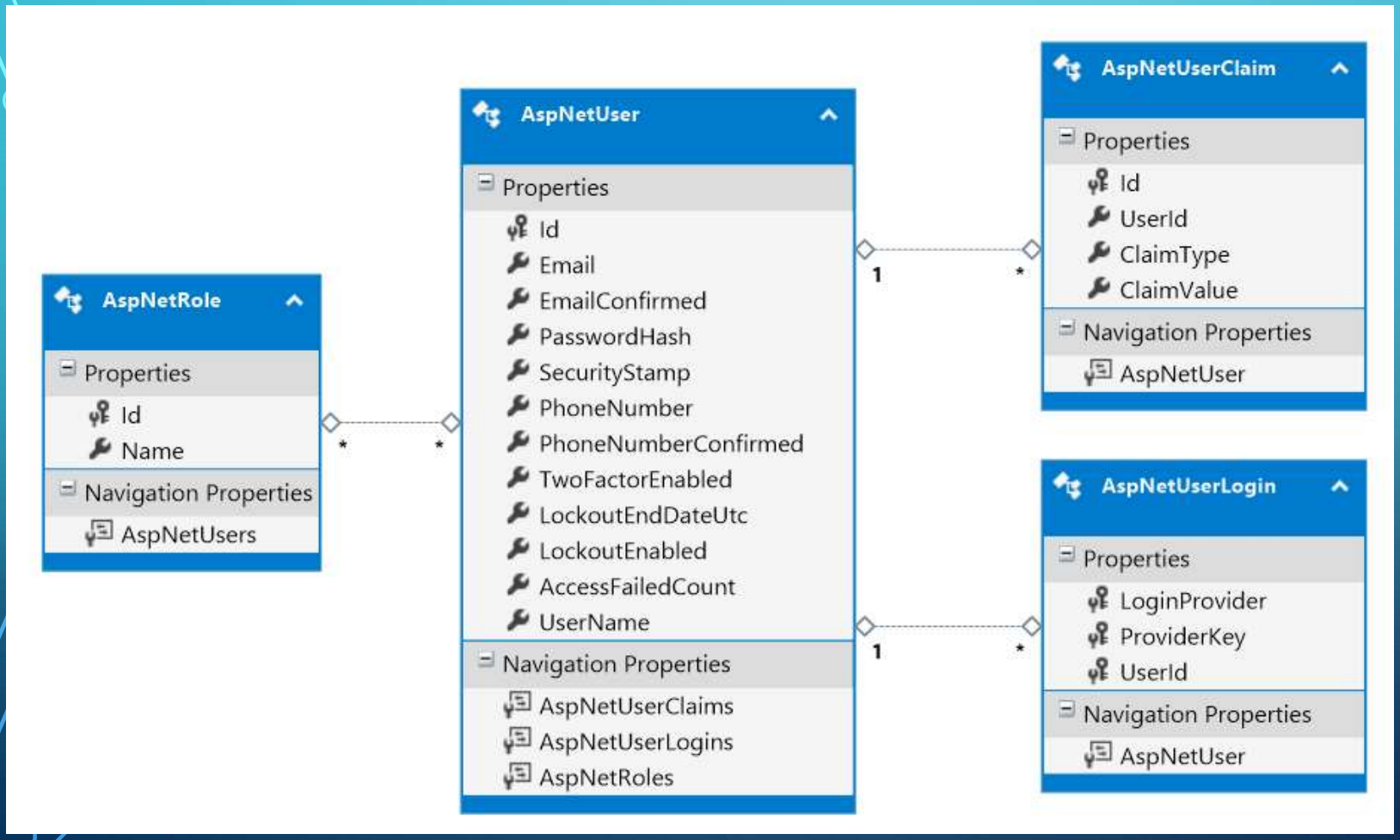


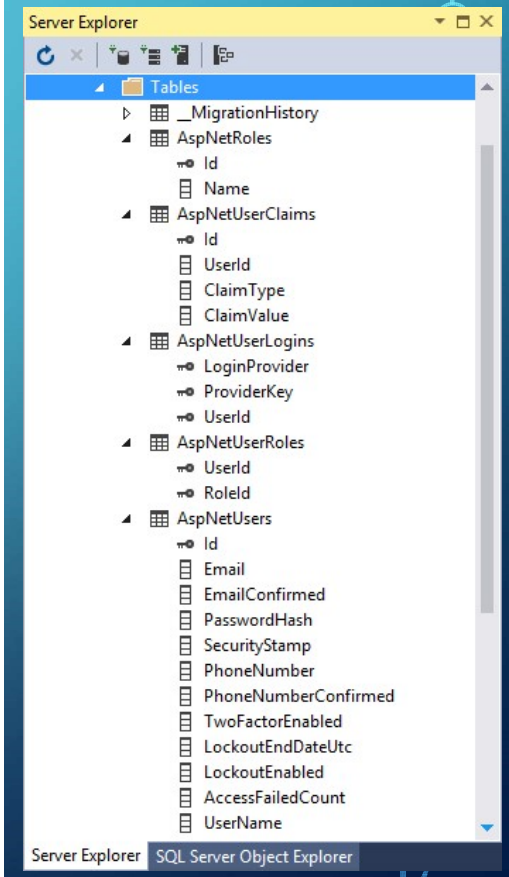
- Geralmente, os dados de identidade do ***ASP.NET Core*** (utilizadores, *passwords*, perfis) são armazenados numa base de dados relacional por meio da ***Entity Framework***





ASP.NET IDENTITY: MODELO DE DADOS







4.2. Validação de Utilizadores

Programação Web – 2024/2025 – LEI D+CET+PL – ISEC/IPC - JB

ESTENDER IDENTITYUSER - APPLICATIONUSER



```
using Microsoft.AspNetCore.Identity;
using System.ComponentModel.DataAnnotations;

namespace AutoWEB.Models
{
    public class ApplicationUser : IdentityUser
    {
        [Display(Name = "Primeiro Nome")]
        9 referências
        public string PrimeiroNome { get; set; }

        [Display(Name = "Último Nome")]
        6 referências
        public string UltimoNome { get; set; }

        [Display(Name = "Data de Nascimento")]
        3 referências
        public DateTime DataNascimento { get; set; }
        3 referências
        public int NIF { get; set; }

        [Display(Name = "A minha fotografia")]
        3 referências
        public byte[]? Fotografia { get; set; }
        0 referências
        public ICollection<Agendamento> Agendamentos { get; set; }
    }
}
```

ASP.NET CORE IDENTITY: AUTORIZE

```
[Authorize]

public ActionResult Index()
{
    return View();
}
```

Filtro

Será chamado pela Framework antes e depois da acção ser exectuada

O atributo *Autorize* irá verificar se o utilizador corrente está autorizado ou não a aceder a este recurso

Se não estiver logado, reencaminha para a página de login



ASP.NET IDENTITY: AUTORIZE



- Aplicar no controlador



```
[Authorize]
public class ProdutosController : ControllerBase
{
    private readonly IProdutoRepository _produtoRepository;

    public ProdutosController(IProdutoRepository
produtoRepository)
    {
        _produtoRepository = produtoRepository;
    }
    [HttpGet]
```

ASP.NET IDENTITY: AUTORIZE

- Aplicar no controlador



```
[Authorize(Roles="Admin, Gestor")]
public class ProdutosController : ControllerBase
{
    private readonly IProdutoRepository _produtoRepository;

    public ProdutosController(IProdutoRepository
produtoRepository)
    {
        _produtoRepository = produtoRepository;
    }
    [HttpGet]
```

ASP.NET IDENTITY: ALLOWANONYMOUS

```
namespace Utilidades.Controllers;

    [AllowAnonymous]

     public class HomeController : Controller
     {
         public ActionResult Index()
         {
             return View();
         }
```



RESTRINGIR AÇÕES A UTILIZADORES

```
[Authorize(Users="joao")]

public ActionResult Create()
{
   return View();
}
```

PERFIS E ROLES INICIAIS



```
using Microsoft.AspNetCore.Identity;
using AutoWEB.Models;
using System;

namespace AutoWEB.Data
{
    4 referências
    public enum Roles
    {
        Admin,
        Formador,
        Cliente
    }
    1 referência
    public static class Inicializacao
    {
        1 referência
        public static async Task CriaDadosIniciais(UserManager<ApplicationUser> userManager, RoleManager<IdentityRole> roleManager)
        {
            //Adicionar defaut Roles
            await roleManager.CreateAsync(new IdentityRole(Roles.Admin.ToString()));
            await roleManager.CreateAsync(new IdentityRole(Roles.Formador.ToString()));
            await roleManager.CreateAsync(new IdentityRole(Roles.Cliente.ToString()));

            //Adicionar Default User - Admin
            var defaultUser = new ApplicationUser
            {
                UserName = "admin@localhost.com",
                Email = "admin@localhost.com",
                PrimeiroNome = "Administrador",
                UltimoNome = "Local",
                EmailConfirmed = true,
                PhoneNumberConfirmed = true
            };
            if (userManager.Users.All(u => u.Id != defaultUser.Id))
            {
                var user = await userManager.FindByEmailAsync(defaultUser.Email);
                if (user == null)
                {
                    await userManager.CreateAsync(defaultUser, "Is3C..00");
                    await userManager.AddToRoleAsync(defaultUser, Roles.Admin.ToString());
                }
            }
        }
    }
}
```



4.3. Tipos de Autenticação e Autorização

Programação Web – 2024/2025 – LEI D+G+PL – ISEC/IPC - JB

TIPOS DE AUTENTICAÇÃO



- **Autenticação em API Web .Net Core 8**

  A autenticação numa API Web .NET é o processo de verificação da identidade de um utilizador ou entidade que está a tentar aceder à API.

  - Garante que apenas os utilizadores autorizados podem aceder aos **endpoints** da API, evitando o acesso não autorizado.
  - O . NET fornece vários métodos e estruturas para implementar a autenticação, garantindo que apenas os utilizadores autorizados podem aceder a recursos específicos ou executar determinadas ações.
  - Os métodos de autenticação comuns incluem os **tokens JWT, OAuth2 e ASP.NET Core Identity**. Estes métodos garantem que apenas os utilizadores autorizados podem aceder a recursos protegidos.

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/JPC/JB

UTILIZAÇÃO DE JSON WEB TOKEN JWT



Tipos de autenticação

1. Autenticação Básica:

- Utiliza um nome de utilizador e palavra-passe.
- As credenciais são enviadas no cabeçalho HTTP e codificadas em Base64.
- Simples, mas não muito seguro, especialmente se não for utilizado em HTTPS.

2. Autenticação baseada em tokens:

- **JWT (JSON Web Tokens):** Muito utilizado para autenticação sem estado. Um token é emitido para um utilizador após o login bem-sucedido, que deve ser enviado com cada pedido subsequente.
- **OAuth:** Um sistema mais complexo que fornece acesso delegado utilizando tokens de acesso. Envolve frequentemente um servidor de autorização e um servidor de recursos.

Programação Web – 2024/2025 – LEI DI-CE+PL – ISEC/IPC - JB

UTILIZAÇÃO DE JSON WEB TOKEN JWT



3. Autenticação baseada em cookies:

- Método tradicional utilizado em aplicações web onde os dados de autenticação são armazenados em cookies.
- Podem ser vulneráveis a **ataques CSRF** se não forem implementados corretamente.

4. Autenticação de chave API:

- Utiliza uma **chave API** enviada no cabeçalho do pedido ou como parâmetro de consulta.
- Simples de implementar, mas não altamente seguro.

5. Autenticação personalizada:

- Esquemas personalizados adaptados a requisitos específicos.
- •Pode combinar vários métodos e verificações de segurança adicionais.

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/IPC - JP



4.4. Utilização de Json Web Token JWT

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/IPC - JB

UTILIZAÇÃO DE JSON WEB TOKEN JWT



Autenticação de API Web .NET com JWT

A autenticação numa API Web .NET utilizando JSON Web Tokens (JWT) é uma abordagem comum para proteger APIs.

- Os JWT são tokens compactos e seguros para URL que podem ser utilizados para transmitir informações de forma segura entre as partes.

Programação Web – 2024/2025 – MEID+CE+PL – ISEC/IPC - JB

UTILIZAÇÃO DE JSON WEB TOKEN JWT



Processo de Autenticação utilizando JWT numa API Web .NET

1. Login de utilizador e geração de tokens

- **Envio de credenciais de utilizador:** O utilizador envia as suas credenciais de login (nome de utilizador e palavra-passe) para o terminal de autenticação da API (por exemplo, POST /api/auth/login).
- **Validação de credenciais:** a API valida as credenciais do utilizador num armazenamento de dados (por exemplo, uma base de dados). Isto normalmente envolve verificar a palavra-passe e verificar se o utilizador existe.
- **Criação de Token:** Se as credenciais forem válidas, o servidor cria um JWT. Este token inclui declarações, que são informações sobre o utilizador (por exemplo, ID do utilizador, roles, etc.).

Programação Web - 2024/2025 - LEI+CET+PL - ISEC/nfc - JB

UTILIZAÇÃO DE JSON WEB TOKEN JWT



2. Estrutura JWT

Um JWT é constituído por três partes:

- **Cabeçalho**
  - **Contém o tipo de token (JWT) e o algoritmo de assinatura (por exemplo, HMAC SHA256).**

```
{
 "alg": "HS256",
 "tipo": "JWT"
}
```

Este JSON é depois codificado em Base64Url para formar a primeira parte do JWT. Base64Url codificado: 'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9'

Programação Web – 2024/2025 – LEI+CE+PL – ISEC/IPC - JB

UTILIZAÇÃO DE JSON WEB TOKEN JWT



- **Pay Load - Carga útil**

Contém os Claims. Estas são declarações sobre o utilizador e dados adicionais. Os claims comuns incluem:

- **sub (subjet): ID do utilizador**
- **exp (expiração): o tempo de expiração do token**
- **iat (issued at - emitido em): o instante em que o token foi emitido**
- **roles: as funções atribuídas ao utilizador**

Programação Web 2024/2025 _ LEI+LD+CE+PL _ ISEC/IPC _ JB

UTILIZAÇÃO DE JSON WEB TOKEN JWT



```
{
 "sub": "1234567890",
 "nome": “Manuel Maria",
 "iat": 1516239022,
 "exp": 1416243327
}
```

Este JSON é depois codificado em Base64Url para formar a segunda parte do JWT.

Base64Url codificado:

‘eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoxNTE2MjM5MDIyfQ’

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/IPC - JB

UTILIZAÇÃO DE JSON WEB TOKEN JWT



- **Assinatura**
  - O cabeçalho codificado e a carga útil são concatenados com um ponto (.) e depois encriptados utilizando o algoritmo especificado e uma chave secreta para produzir a assinatura. Isto garante a integridade e autenticidade do token.
    - A assinatura é utilizada para verificar se o remetente do JWT é quem diz ser e para garantir que a mensagem não foi alterada ao longo do caminho.

```
HMACSHA256(
    base64UrlEncode(cabeçalho) + "." + base64UrlEncode(payload),
    secret
)
```

Resultando em algo como: 'SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c'

Programação Web – 2024/2025 – LEI D+CF+PL – ISEC/IPC – JB

UTILIZAÇÃO DE JSON WEB TOKEN JWT



Combinam-se todas as partes para obter o JWT:

O cabeçalho codificado, a carga útil e a assinatura são combinados numa única string, separada por pontos.

(cabeçalho.payload.assinatura)

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9

.

eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoxNTE2MjM5MDIyfQ

.

SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c

Programação Web – 2024/2025 – LEI D+ME+PL – ISEC/IPC – JB

UTILIZAÇÃO DE JSON WEB TOKEN JWT



3. Transmissão do token

- O JWT gerado é **enviado para o cliente** (geralmente na resposta de um corpo de resposta de pedido de login ou num cabeçalho de resposta).

Programação Web – 2024/2025 – LEI, D+CE+PL – ISEC/IPC - JB

UTILIZAÇÃO DE JSON WEB TOKEN JWT



Swagger UI

https://localhost:7250/swagger/index.html

Responses

Curl

```
curl -X 'POST' \
  'https://localhost:7250/api/Utilizadores/LoginUser' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
  "email": "admin@localhost.com",
  "password": "Is3C..00"
}'
```

Request URL

```
https://localhost:7250/api/Utilizadores/LoginUser
```

Server response

Code	Details
200	Response body

```
{
  "accesstoken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJodHRwOi8vc2NoZW1hcy54bWxzb2Fwlm9yZy93cy8yMDA1LzA1L2lkZW50aXR5L2NsYWltcy9lbWFpbGFkZHJlc3MiOiJhZG1pbkBsb2NhbGhvc3QuY29tIiwiaHR0cDovL3NjaGVtYXMubWljcm9zb2Z0LmNvbS93cy8yMDA4LzA2L2lkZW50aXR5L2NsYWltcy9yb2xlIjoiQWRtaW4iLCJleHAiOjE3MzcxMTM4MzMsImlzcyI6Imh0dHBzOi8vbG9jYWxob3N0OjcwNjYvIiwiYXVkIjoiaHR0cHM6Ly9sb2NhbGhvc3Q6NzA2Ni8ifQ.DFQVfPQ9WYtUUk8oiUIX8Sawm14klNAj_Xf0fslwpRE",
  "tokentype": "bearer",
  "utilizadorid": "4b5a3028-0e2f-4f66-9e1e-4d0fe47b6e4d",
  "utilizadornome": "Administrador"
}
```

Download

Response headers

```
cache-control: no-cache,no-store
content-type: application/json; charset=utf-8
date: Wed,18 Dec 2024 11:37:13 GMT
expires: Thu,01 Jan 1970 00:00:00 GMT
pragma: no-cache
server: Kestrel
```

Responses

Code	Description	Links
200	OK	*No links*

UTILIZAÇÃO DE JSON WEB TOKEN JWT



4. Armazenamento pelo Cliente do Token

O cliente armazena o JWT, normalmente em armazenamento local, armazenamento numa Session, armazenamento em memória ou HTTP cookies.

- A forma recomendada de armazenar JWT é com HTTP cookies. Este é o método mais seguro, pois impede o acesso ao token via JavaScript, mitigando assim os ataques XSS.
- Combinar com os tokens CSRF: ao utilizar cookies, normalmente também se implementa a proteção CSRF para mitigar os ataques CSRF.

Configurar o cookie (do lado do servidor)

Do lado do servidor, pode-se definir um HTTP cookie contendo o token JWT. Isto torna-o inacessível ao JavaScript executado no lado do cliente.

Programação Web – 2024/2025 – LEI LtCE+PL – ISEC/IPC – JB

UTILIZAÇÃO DE JSON WEB TOKEN JWT



Configurar o cookie (do lado do servidor)

- Do lado do servidor, pode-se definir um HTTP cookie contendo o token JWT. Isto torna-o inacessível ao JavaScript executado no lado do cliente.

Recuperando o token

- Os HTTP cookies são enviados automaticamente com cada pedido para o servidor, pelo que não é preciso recuperá-los manualmente.

Programação Web – 2024/2025 – LEI/D+CE/PL – ISEC/IPC - JB

UTILIZAÇÃO DE JSON WEB TOKEN JWT



5.Solicitações subsequentes por parte da API

- Para pedidos subsequentes para endpoints de API protegidos, o cliente inclui o JWT no cabeçalho Authorization do pedido HTTP, formatado como Bearer <token>. Isto garante que o servidor pode validar o token e verificar se o pedido está autorizado.
- Assim que tiver o token JWT (do endpoint de login), este é incluido no cabeçalho Authorization dos pedidos subsequentes para os endpoints protegidos.

Programação Web – 2024/2025 – LEI, D+CE+LTI – ISEC/PC - JB

UTILIZAÇÃO DE JSON WEB TOKEN JWT



6. Validação de tokens do lado do servidor

- **Extração de tokens:** a API extrai o JWT do cabeçalho de autorização.
- **Análise de Token:** O token extraído é analisado para descodificar o seu conteúdo. Para JWTs, isto envolve normalmente dividir o token nas suas três partes (cabeçalho, carga útil e assinatura) e descodificar a carga JSON codificada em Base64Url para obter as claims.
- **Verificação de Assinatura:** Se o token for um JWT (JSON Web Token), o servidor valida a sua assinatura criptográfica. Isto envolve:
- **Recuperar a chave pública** (se estiver a utilizar criptografia assimétrica) ou a chave secreta (se estiver a utilizar criptografia simétrica) que foi utilizada para assinar o token.

Programação Web – 2024/2025 – LEI,D+CE,PL – ISEC/IPC – JB

UTILIZAÇÃO DE JSON WEB TOKEN JWT



- **Verificar a assinatura digital** para garantir que o token não foi adulterado desde que foi emitido

- **Validação das claims:** Após validar a assinatura, o servidor verifica as claims do token. Isto inclui informações como a expiração do token (exp), o emissor (iss), o público (aud) e quaisquer declarações personalizadas relevantes para a sua aplicação. O servidor verifica se estas declarações estão presentes, corretas e dentro de intervalos aceitáveis (por exemplo, o token não expirou).
- **Autorização do utilizador:** Depois de o token ser validado com sucesso, o servidor utilizará as informações do token (por exemplo, **ID do utilizador, roles**) para autorizar o pedido.
  - A autorização implica verificar se o utilizador autenticado possui as permissões necessárias para aceder ao recurso solicitado ou executar a ação solicitada.

Programação Web – 2024/2025 – LEI-LECEI – ISEC/IPC - JB

UTILIZAÇÃO DE JSON WEB TOKEN JWT



7.º Conceder acesso ou negar

- **Se o token for válido e o utiliz4040ador estiver autorizado, a API processa o pedido e devolve a resposta adequada.**

- **Se o token for inválido ou o utilizador não estiver autorizado, a API devolve uma resposta de erro adequada (e.g., 401 Unauthorized or 403 Forbidden).**

Programação Web – 2024/2025 – LEI+D+CE+RI – ISEC/IPC - JB